

Probeklausur 06.03.2026

Aufgabe 1 (47.25 Punkte)

Implementiere die Klassen

- **Boardable** (1.5 Punkte),
- **TicketClass** (3.5 Punkte),
- **Passenger** (3.5 Punkte),
- **Seat** (10.0 Punkte),
- **BusinessSeat** (5.75 Punkte),
- **EconomySeat** (6.0 Punkte),
- **Airplane** (12.0 Punkte) und
- **ExamTask** (5.0 Punkte)

entsprechend dem Klassendiagramm.

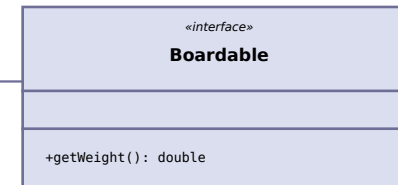
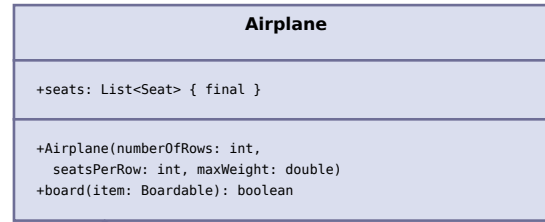
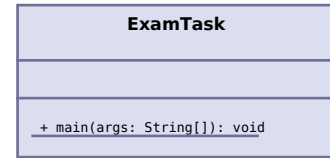
Befolge alle Hinweise bei der Implementierung!

Die Klasse CrewMember ist gegeben und muss nicht implementiert werden.

Glossar

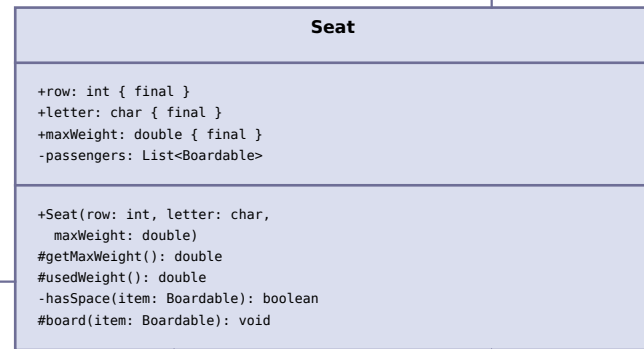
Englisch	Deutsch
airplane	Flugzeug
board	einsteigen
boardable	einsteigefaeig
crew member	Besatzungsmitglied
letter	Buchstabe
passenger	Passagier
row	Reihe
seat	Sitzplatz
ticket class	Ticketklasse
weight	Gewicht

Klassendiagramm



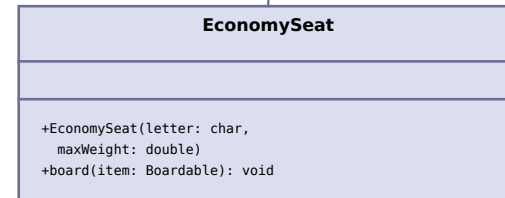
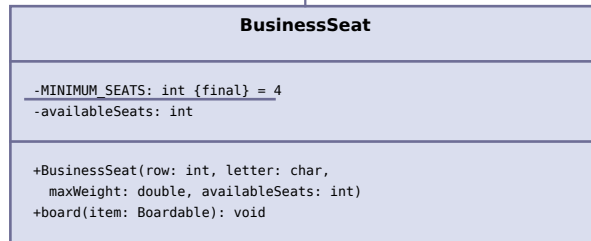
implements

implements



extends

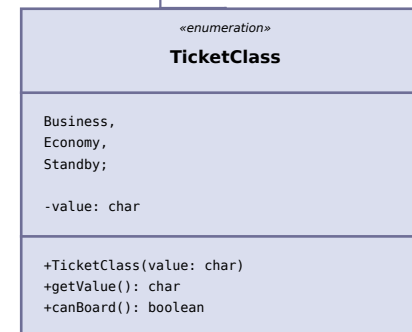
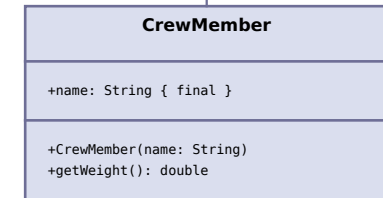
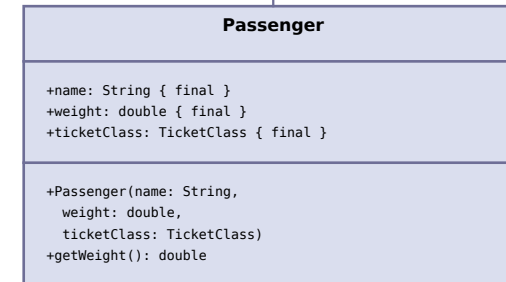
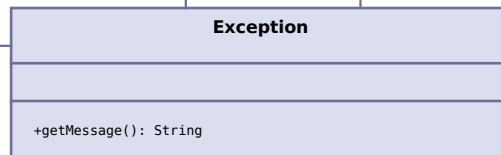
extends



throws

throws

throws



Hinweise zur Klasse Boardable (Einsteigefähig)

- Die Methode **getWeight** soll das Gewicht zurückgeben.

Hinweise zur Klasse TicketClass (Ticketklasse)

- Die Methode **getValue** soll den Wert (value) der Ticketklasse zurückgeben.
- Die Methode **canBoard** soll true zurückgeben, wenn abhängig von der Ticketklasse eingestiegen werden darf.

Es darf eingestiegen werden, sofern es kein Standby-Ticket ist.

Hinweise zur Klasse Passenger (Passagier)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getWeight** soll das Gewicht zurückgeben.

Das Gewicht eines Passagiers ist die Hälfte des weight-Attributs.

Hinweise zur Klasse Seat (Sitzplatz)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getMaxWeight** soll das maximale Gewicht eines Sitzplatzes berechnen.
Das maximale Gewicht eines Sitzplatzes ist abhängig von maxWeight.
- Die Methode **usedWeight** soll das bereits benutzte Gewicht eines Sitzplatzes berechnen.
Das benutzte Gewicht ist die Summe des Gewichts aller Passagiere eines Sitzplatzes.
- Die Methode **hasSpace** soll ermitteln, ob der eingehende Passagier auf den Sitz passt.

Ein Passagier hat genügend Platz, sobald das Gewicht des eingehenden Passagiers kleiner ist als das restlich vorhandene Gewicht eines Sitzplatzes.

- Die Methode **board** soll versuchen einen Passagier auf dem Sitz zu platzieren.

Hat der zu platzierende Passagier genug Platz, soll er dem Sitz hinzugefügt werden. Andernfalls soll ein Fehler ausgelöst werden, dass kein Platz mehr vorhanden ist.

Hinweise zur Klasse **BusinessSeat**

- **Konstruktor** soll alle Attribute initialisieren.

Der Business-Sitz hat breitere Sitze und somit 10 Einheiten weniger maximales Gewicht.

- Die Methode **board** soll versuchen, einen Passagier auf dem Sitz zu platzieren.

Nur Besatzungsmitglieder (CrewMember) dürfen auf dem Business-Sitz platziert werden. Ist die Anzahl der verfügbaren Sitze (availableSeats) kleiner als die Mindestanzahl (MINIMUM_SEATS), darf niemand auf dem Sitz platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgelöst werden, dass nur Crew in der Business Klasse erlaubt ist.

Hinweise zur Klasse **EconomySeat**

- **Konstruktor** soll alle Attribute initialisieren.

Der Economy-Sitz soll immer in der untersten Reihe sein (Reihe 1).

- Die Methode **board** soll versuchen, einen Passagier auf dem Sitz zu platzieren.

Nur Passagiere (Passenger) deren Ticketklasse einsteigen darf, dürfen auf dem Economy-Sitz platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgelöst werden, dass nur Passagiere mit gültigem Ticket erlaubt sind.

Hinweise zur Klasse Airplane (Flugzeug)

- **Konstruktor** soll alle Attribute initialisieren.

Jedes Flugzeug hat mehrere Sitze. Es gibt jedoch immer genau einen Business-Sitz und einen Economy-Sitz. Der Konstruktor soll nach der Initialisierung aller Attribute zuerst den Business-Sitz, dann den Economy-Sitz und anschliessend die normalen Sitze erzeugen und der internen Liste hinzufügen.

Die normalen Sitze werden in einem verschachtelten Loop erzeugt: Der äussere Loop iteriert über die Reihen (absteigend), der innere Loop über die Sitzbuchstaben (A, B, C, ...).

Der Economy-Sitz hat immer die geringste Reihe (Reihe 1) und der Business-Sitz immer die höchste Reihe. Der Business-Sitz hat 6 verfügbare Sitze (availableSeats).

Beispiel Liste mit numberOfRows = 3 und seatsPerRow = 2:

```
- BusinessSeat - Reihe = 5, Buchstabe = A
- EconomySeat - Reihe = 1, Buchstabe = A
- normaler Sitz - Reihe = 4, Buchstabe = A
- normaler Sitz - Reihe = 4, Buchstabe = B
- normaler Sitz - Reihe = 3, Buchstabe = A
- normaler Sitz - Reihe = 3, Buchstabe = B
- normaler Sitz - Reihe = 2, Buchstabe = A
- normaler Sitz - Reihe = 2, Buchstabe = B
```

- Die Methode **board** soll einen Passagier platzieren und zurückgeben, ob es funktioniert hat.

Versuche einen Platz für den Passagier im Flugzeug zu finden. Wenn ein Platz gefunden wurde, soll auf der Konsole die Reihe und der Buchstabe ausgegeben werden.

Wenn ein Passagier auf einem Sitz nicht platziert werden konnte, soll der Grund auf der Konsole ausgegeben werden.

Hinweise zur Klasse ExamTask

Es soll ein Flugzeug mit 4 Reihen, 3 Sitzen pro Reihe und einem maximalen Gewicht von 90 erstellt werden.

Erstelle ein Besatzungsmitglied mit einem Namen deiner Wahl und versuche es im Flugzeug zu platzieren.

Iteriere über alle Ticketklassen. Erstelle für jede Ticketklasse einen Passagier mit einem Gewicht von 70 und versuche ihn im Flugzeug zu platzieren. Verwende den Wert (getValue) der Ticketklasse im Namen des Passagiers.

Falls kein Platz gefunden wurde, soll dies zusammen mit dem Wert der Ticketklasse auf der Konsole ausgegeben werden.

Java API

Klasse	Methode	Statisch	Rueckgabotyp
TicketClass	values()	X	TicketClass[]
TicketClass	name()		String
Exception	getMessage()		String
Boolean	valueOf(s: String)	X	Boolean
Boolean	valueOf(b: boolean)	X	Boolean
Double	valueOf(s: String)	X	Double
Double	valueOf(d: double)	X	Double
Integer	valueOf(s: String)	X	Integer
Integer	valueOf(i: int)	X	Integer
String	charAt(index: int)		char
String	length()		int

Java Collections Framework

Klasse	Methode	Statisch	Rueckgabotyp
ArrayList<T>	add(element: T)		boolean
ArrayList<T>	add(index: int, element: T)		void
ArrayList<T>	contains(element: T)		boolean
ArrayList<T>	get(index: int)		T
ArrayList<T>	remove(index: int)		T
ArrayList<T>	remove(element: T)		boolean
ArrayList<T>	size()		int
Collections	sort(list: List<T>)	X	void
Collections	sort(list, c: Comparator<T>)	X	void

Schnittstellen

Klasse	Methode	Statisch	Rueckgabotyp
Comparable<T>	compareTo(o: T)		int
Comparator<T>	compare(o1: T, o2: T)		int